Browser Automation Capability Specification

Project NOVA Web Automation, Session Management, and Browser Abstraction Interfaces

---

Field	Value
Document ID	NOVA-SPEC-006
Version	1.0
Status	`APPROVED`
Author	Antigravity (Lead Software Engineering Agent)
Reviewer	ChatGPT (Chief Architect)
Approved By	Praveen (Project Founder)
Created	2026-06-28
Last Updated	2026-06-28
Dependencies	NOVA-SPEC-001, NOVA-SPEC-002, NOVA-SPEC-003

Revision History

Version	Date	Author	Summary of Changes
1.0	2026-06-28	Antigravity	Consolidate Browser Overview, Tab Control, Page Interaction, Web Observation, Security, and Provider Abstraction

specifications.

Table of Contents

Purpose & Scope

This specification defines the functional boundaries, API contracts, and security policies for the **Browser Automation Capability**. It enables NOVA to programmatically launch browsers, open URLs, click web elements, input form details, and parse pages using replaceable drivers.

Browser Lifecycle Management

- **Launch Browser:** Starts a Chromium, Firefox, or Webkit browser instance.
- **Close Browser:** Safely terminates browser handles and releases resources.
- **Profile Isolation:** Launches using specified profiles or temp contexts to persist cookies and logins.
- **Session Restoration:** Reloads open tabs and cookie maps from history JSON structures.

Tab & Window Orchestration

- **Tab Control:** Opens, closes, pins, or switches active tabs.
- **Window Control:** Focuses active windows and resizes bounds.
- **Metadata Access:** Reads active URL targets, page titles, and loads states.

Page Element Interaction

Provides programmatic actions on page HTML nodes:

- **Click Element:** Emulates clicks on selectors.
- **Fill Forms:** Selects input blocks and inputs values.
- **Scroll Page:** Scrolls coordinates or shifts elements into view.
- **File Transfers:** Programmatically triggers file uploads and downloads.
- **Dialog Handlers:** Accepts or cancels Javascript popups automatically.
- **Wait Conditions:** Blocks execution loops until targets appear in the DOM.

Web Observation & DOM Parsing

- **DOM Trees:** Extracts page HTML and raw DOM structures.
- **Text Scrapes:** Reads page texts and lists links.
- **Visual Snaps:** Captures page screenshots.
- **Observer Pipeline:** Normalizes layouts and coordinates to feed page updates to the Vision Engine.

Security & Sensitive Domain Policies

To secure browser integrations, the subsystem enforces these rules:

9. **Credential Protection:** Password values typed into forms must be obfuscated in logs and memory.
10. **Sensitive Domain Blocks:** Confirms actions via explicit permissions before navigating to banking, e-commerce, or credential vault domains.
11. **Upload Directories Validation:** Asserts upload file paths are within allowed workspace folders.

Provider Abstraction Interface

To prevent lock-in to Playwright or Selenium, the capability interacts with browser drivers using the `IBrowserProvider` interface:

```
class IBrowserProvider(ABC):
    @abstractmethod
    def launch(self, browser_type: str, headless: bool, profile: Optional[str] =
None) -> str: ...
    @abstractmethod
    def close(self) -> None: ...
    @abstractmethod
    def navigate(self, url: str) -> None: ...
    @abstractmethod
```

```
    def click_element(self, selector: str) -> None: ...
    @abstractmethod
    def fill_input(self, selector: str, text: str) -> None: ...
    @abstractmethod
    def read_dom(self) -> str: ...
    @abstractmethod
    def take_screenshot(self) -> bytes: ...
```

Testing & Verification Targets

- **Cross-Browser Integration:** Pytest runners execute tests across Chrome, Edge, and Firefox profiles.
- **Headed/Headless Assertions:** Test verification runs tests in headless environments during CI/CD, and headed runs locally.
- **Form & Dialog Tests:** Checks form inputs and custom alert dismissals.